

GODOT
Game engine

L'Eserciziario

Corso di Godot 

Versione 0.4 · 26/07/2026

Corso a cura del prof. Nicola Regge

Come funziona ogni esercizio (leggere prima!)

Ogni esercizio ha **4 livelli di aiuto**. Prova sempre da solo, e apri il livello successivo **solo se sei bloccato**:

1.  **Descrizione** — cosa devi ottenere.
2.  **Aiuto** — un indizio su come fare.
3.  **La scena** — quali nodi creare (i “mattoncini”).
4.  **Codice completo** — la soluzione da copiare/incollare.

Regola: se copi il **Codice completo**, poi devi saper **spiegare a voce cosa fa, riga per riga**. Se lo sai spiegare, hai imparato lo stesso. 😊

(Nel file `.md` i livelli 2–4 sono a scomparsa: clicca sul triangolino per aprirli. Nel PDF sono già aperti.)

 **Oltre agli esercizi numerati ci sono i “Progetti BOSS”**: giochi già pronti, più grossi, che non si copiano riga per riga — si **aprono, si giocano e si rendono propri** (cambi colori, titolo, ci metti una tua foto). Sono il premio: la cosa figa da mostrare subito agli amici. Il codice è già nel repository, lo capiremo un pezzo alla volta.

 **Nota per il docente**: per ora gli esercizi sono raccolti così come nascono, non in ordine di difficoltà. Più avanti li riordineremo (per difficoltà o per quando si svolgono in classe). L'importante adesso è raccogliarli.

ESERCIZIO 1

Il bottone che saluta 🖐️

Ponte da Lazarus: è il tuo `Button1Click` che cambia una `Caption`!

🟢 Descrizione

Crea una schermata con **un bottone** e **una scritta**. Quando premi il bottone, la scritta deve cambiare (es. da “...” a “Ciao! Mi hai premuto”).

🎨 **Fallo tuo:** scegli **tu** la frase del saluto e il **colore** della scritta — così il gioco è già tuo. Nessuno lo farà uguale al tuo!

🟡 Aiuto

- In Godot il bottone è il nodo **Button**, la scritta è il nodo **Label**.
- La proprietà `text` di Godot è come la **Caption** di Lazarus.
- L’evento “click” in Godot si chiama **segnale** `pressed`. Lo colleghi a una tua funzione con `bottone.pressed.connect(la_mia_funzione)`.

🟠 La scena (i nodi da creare)

1. Nodo radice: **Node2D** (rinominalo `Main`).
2. Figlio: **Button** → rinominalo `BottoneCiao`.
3. Figlio: **Label** → rinominalo `Etichetta`.
4. Attacca uno **script** al nodo radice `Main`.

Codice completo

```
extends Node2D
```

```
# $NomeNodo = prende un nodo figlio per nome (come Button1 in Lazarus)
```

```
@onready var bottone: Button = $BottoneCiao
```

```
@onready var etichetta: Label = $Etichetta
```

```
func _ready() -> void:
```

```
# Posizioniamo i due elementi così non si sovrappongono
```

```
bottone.position = Vector2(100, 100)
```

```
bottone.text = "Salutami!" # <- come Button.Caption in Lazarus
```

```
etichetta.position = Vector2(100, 180)
```

```
etichetta.text = "..."
```

```
# 🎨 FALLO TUO: scegli il colore della scritta (i numeri sono rosso, verde, blu)
```

```
etichetta.add_theme_color_override("font_color", Color(1, 0, 0)) # rosso
```

```
# Colleghiamo il "click" (segnale pressed) alla nostra funzione
```

```
bottone.pressed.connect(_quando_premo)
```

```
# Questa e' come il tuo Button1Click di Lazarus
```

```
func _quando_premo() -> void:
```

```
# 🗨️ FALLO TUO: scrivi qui il TUO saluto
```

```
etichetta.text = "Ciao! Mi hai premuto."
```

ESERCIZIO 2

Muovi il quadrato



Concetto nuovo: il game loop, cioè `_process`.

Descrizione

Fai comparire un **quadrato** che puoi muovere in tutte le direzioni con le **freccie** della tastiera.

Aiuto

- Un quadrato colorato semplice = nodo **ColorRect**.
- Il movimento va scritto in `_process(delta)`: e' la funzione che gira ~60 volte al secondo (in Lazarus non c'era: il programma stava fermo).
- Le frecce si leggono con `Input.is_action_pressed("ui_left")` (e `ui_right`, `ui_up`, `ui_down`).
- Moltiplica sempre la velocita' per `delta`, cosi' va uguale su ogni PC.

La scena (i nodi da creare)

1. Nodo radice: **Node2D** (rinominalo `Main`).
2. Figlio: **ColorRect** → rinominalo `Quadrato`.
3. Attacca uno **script** al nodo radice `Main`.

Codice completo

```
extends Node2D

@onready var quadrato: ColorRect = $Quadrato
const VELOCITA: float = 300.0 # pixel al secondo

func _ready() -> void:
    quadrato.size = Vector2(60, 60)
    quadrato.color = Color(0.3, 0.7, 1.0) # azzurro
    quadrato.position = Vector2(200, 200)

# _process gira a OGNI fotogramma: qui muoviamo il quadrato
func _process(delta: float) -> void:
    if Input.is_action_pressed("ui_left"):
        quadrato.position.x -= VELOCITA * delta
    if Input.is_action_pressed("ui_right"):
        quadrato.position.x += VELOCITA * delta
    if Input.is_action_pressed("ui_up"):
        quadrato.position.y -= VELOCITA * delta
    if Input.is_action_pressed("ui_down"):
        quadrato.position.y += VELOCITA * delta
```

ESERCIZIO 3

Prendi la moneta

Mette insieme: movimento + oggetto che cade + punteggio. Verso il gioco vero.

Descrizione

Un **cestino** in basso (che muovi con le frecce $\leftarrow \rightarrow$) e una **moneta** che cade dall'alto. Se la prendi col cestino fai **+1 punto** e la moneta riparte dall'alto in una colonna a caso. Mostra il punteggio a schermo.

Aiuto

- Cestino e moneta: due **ColorRect**. Il punteggio: un **Label**.
- Muovi il cestino in `_process` (come nell'Esercizio 2, ma solo sinistra/destra).
- Fai scendere la moneta ogni fotogramma: `moneta.position.y += velocita * delta`.
- Per capire se il cestino "tocca" la moneta usa i rettangoli: `Rect2(a.position, a.size).intersects(Rect2(b.position, b.size))`.
- Quando la moneta esce sotto (o e' presa), rimettila in alto a una `x` a caso.

La scena (i nodi da creare)

1. Nodo radice: **Node2D** (rinominalo `Main`).
2. Figlio **ColorRect** → `Cestino`.
3. Figlio **ColorRect** → `Moneta`.
4. Figlio **Label** → `Punteggio`.
5. Attacca uno **script** al nodo radice `Main`.

extends Node2D

@onready var cestino: ColorRect = \$Cestino

@onready var moneta: ColorRect = \$Moneta

@onready var punteggio: Label = \$Punteggio

const VELOCITA_CESTINO: float = 500.0

const VELOCITA_MONETA: float = 300.0

var punti: int = 0

var larghezza: float

func _ready() -> **void**:

 larghezza = get_viewport_rect().size.x

Cestino in basso

 cestino.size = Vector2(120, 24)

 cestino.color = Color(0.6, 0.4, 0.2)

 cestino.position = Vector2(larghezza / 2.0 - 60, get_viewport_rect().size.y - 60)

Moneta

 moneta.size = Vector2(30, 30)

 moneta.color = Color(1.0, 0.85, 0.1)

 _rimetti_in_alto()

Punteggio

 punteggio.position = Vector2(20, 20)

 _aggiorna_punteggio()

func _process(delta: float) -> **void**:

Muovi il cestino

if Input.is_action_pressed("ui_left"):

 cestino.position.x -= VELOCITA_CESTINO * delta

if Input.is_action_pressed("ui_right"):

 cestino.position.x += VELOCITA_CESTINO * delta

 cestino.position.x = clamp(cestino.position.x, 0, larghezza - cestino.size.x)

Fai scendere la moneta

 moneta.position.y += VELOCITA_MONETA * delta

Presa?

if Rect2(cestino.position, cestino.size).intersects(Rect2(moneta.position, moneta.size)):

 punti += 1

 _aggiorna_punteggio()

 _rimetti_in_alto()

Persa (uscita sotto)?

elif moneta.position.y > get_viewport_rect().size.y:

 _rimetti_in_alto()

func _rimetti_in_alto() -> **void**:

var x := randf_range(0, larghezza - moneta.size.x)

 moneta.position = Vector2(x, -moneta.size.y)

```
func _aggiorna_punteggio() -> void:  
    punteggio.text = "Monete: %d" % punti
```

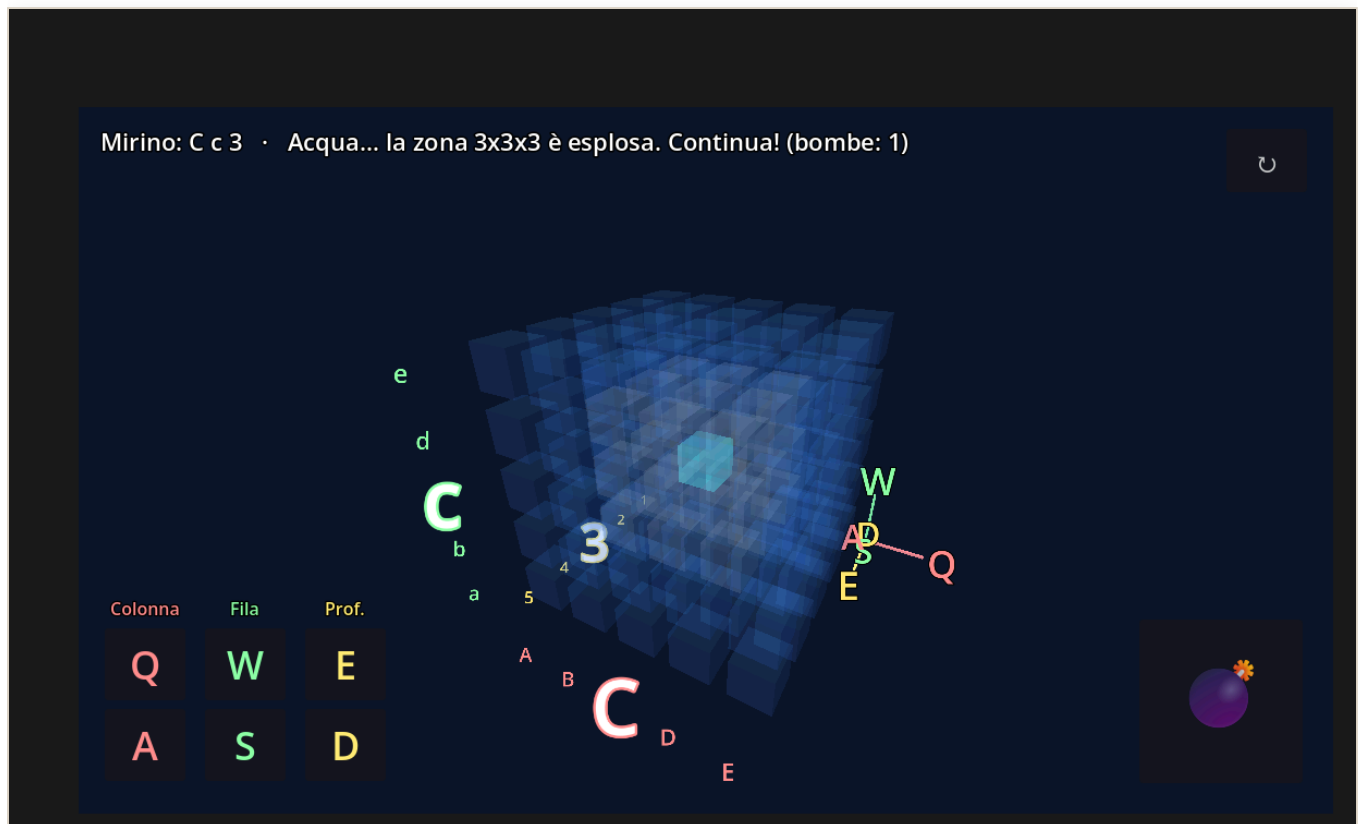
ESERCIZIO BOSS

Affonda la Bonomi

Il primo “progetto boss”: una battaglia navale in 3D già giocabile. Si apre, si gioca, si rende proprio.

Descrizione

La solita battaglia navale, ma **in tre dimensioni**: al posto della griglia a righe e colonne c'è un **cubo** di celle d'acqua. Dentro è nascosto **un sottomarino**. Muovi un **mirino** e lanci una **bomba di profondità**: esplode e colpisce una zona **3×3×3** attorno al punto. Se il sottomarino è lì → **COLPITO!** Questo gioco è **già fatto**: il tuo compito è **aprirlo, giocarci e farlo tuo**.



Affonda la Bonomi in azione: il cubo d'acqua con il mirino verde, le coordinate scritte attorno al cubo e, in basso a sinistra, i comandi colorati dei tre assi (Colonna Q/A, Fila W/S, Profondità E/D).

Aiuto — come aprirlo e giocarci

1. [APP – Godot] finestra iniziale (Gestore progetti), in alto a destra **Importa** → scegli la cartella **battaglia-navale-3d** e il file **project.godot** → **Importa e modifica**.
2. Premi **F5** (Esegui). All'avvio scegli **quanti cubi per lato** (da **4**, facilissimo, fino a **10**, difficile).
3. Comandi (le lettere sono disposte come **tre colonne della tastiera**):
4. **Q / A** = Colonna (rosso) · **W / S** = Fila (verde) · **E / D** = Profondità (giallo)
5. **SHIFT + le stesse lettere** = gira il cubo · **dito/mouse trascinato** = gira il cubo
6. **SPAZIO** = lancia la bomba · **U / INVIO** = rigioca (richiede di nuovo la difficoltà)

Fallo tuo (la parte più importante!)

Apri **battaglia-navale-3d/main.gd** e cambia queste cose per rendere il gioco **tuo** (dopo ogni modifica premi **F5** e guarda l'effetto):

- **I colori dell'acqua e del mirino:** in alto trovi righe tipo `const COL_ACQUA := Color(...)` e `const COL_MIRINO := Color(...)`. Cambia i tre numeri (rosso, verde, blu, da 0 a 1) e avrai il **tuo** stile.
- **La tua foto al posto di Serena:** metti un file **serena.jpg** (una tua foto o un meme) nella cartella **battaglia-navale-3d/**: comparirà quando affondi il sottomarino, lampeggiando con il teschio dei pirati.
- **Il titolo del gioco:** in **project.godot**, alla voce `config/name="..."`, scrivi il **nome che vuoi tu**.
- **La difficoltà di partenza / la potenza della bomba:** prova a cambiare **RAGGIO_BOMBA** (1 = zona 3×3×3; 2 = zona 5×5×5, molto più potente).

 **Mostralo:** quando l'hai personalizzato, fai una partita davanti a un compagno. “Questo l'ho fatto **io**” vale più di qualsiasi voto.

● Il codice completo (dov'è e com'è fatto)

Il codice **c'è già tutto** ed è versionato nel repository, nel file `battaglia-navale-3d/main.gd` (circa 600 righe). Non va copiato a mano: è il nostro **progetto boss**, lo leggeremo **un pezzo alla volta**.

Le idee sono le **stesse degli esercizi precedenti**, portate in 3D:

- il **game loop** `_process(delta)` per girare il cubo (come nell'Esercizio 2);
- **leggere i tasti** con `Input.is_key_pressed(...)` (come nel muovere il quadrato);
- **costruire tutto da codice** (celle, luci, telecamera, bottoni) invece che a mano.

Regola d'oro (vale anche qui): se sai **spiegare a voce** cosa fa un pezzo di codice, quel pezzo è tuo. Partiremo dai pezzi più facili (i colori, i comandi) e saliremo piano piano.

Changelog dell'eserciziario

Versione	Data	Cosa e' cambiato
0.1	26/07/2026	Prima stesura: Es.1 (bottone/Caption, ponte da Lazarus), Es.2 (game loop, muovi il quadrato), Es.3 (prendi la moneta: movimento+caduta+punteggio). Formato a 4 livelli di aiuto.
0.2	26/07/2026	Introdotti i "Progetti BOSS" (giochi pronti da personalizzare). Aggiunto l'Esercizio BOSS "Affonda la Bonomi" (battaglia navale 3D): apri · gioca · fallo tuo · il codice nel repository.
0.3	26/07/2026	Aggiunto lo screenshot del gioco "Affonda la Bonomi" nell'Esercizio BOSS.
0.4	26/07/2026	Stile: sottotitoli degli esercizi senza parentesi (titolo + sottotitolo centrato più piccolo); blocchi di codice nero-su-bianco (fondo chiaro) per stampare senza sprecare toner.